

Deliverables for BZT/BZA

01 Requirements documentation

Use case diagram and descriptions for the game

Tabellarische Beschreibung der Anwendungsfälle (Use Case)

UC1: Spiel starten

Aspekt	Beschreibung
Name	Spiel starten
Akteur	Spieler
Beschreibung	Der Spieler startet das Memory-Spiel
Auslöser	Spieler öffnet die Anwendung oder wählt "Neues Spiel"
Vorbedingungen	Keine
Nachbedingungen	Das Hauptmenü wird angezeigt
Standardablauf	1. Spieler startet die Anwendung 2. Willkommensbildschirm wird angezeigt 3. Hauptmenü wird geladen
Alternative Abläufe	Wenn das Spiel bereits läuft: Bestätigung zum Beenden des aktuellen Spiels

UC2: Thema auswählen

Aspekt	Beschreibung
Name	Thema auswählen
Akteur	Spieler
Beschreibung	Der Spieler wählt eines der drei Themen: Mathematik, Englisch oder Wörter
Auslöser	Spieler klickt auf die Themenauswahl

Vorbedingungen	Das Spiel wurde gestartet
Nachbedingungen	Thema wurde ausgewählt und entsprechende Kartensets werden geladen
Standardablauf	1. Spieler wählt eines der drei Themen aus 2. System lädt die entsprechenden Karten für das gewählte Thema
Alternative Abläufe	Spieler kann das Spiel beenden oder direkt mit dem voreingestellten Thema das Spiel starten

UC3: Karten umdrehen

Aspekt	Beschreibung
Name	Karten umdrehen
Akteur	Spieler
Beschreibung	Der Spieler dreht einzelne Karten um, um deren Inhalt zu sehen
Auslöser	Spieler klickt oder berührt auf eine verdeckte Karte
Vorbedingungen	Das Spiel wurde gestartet und die Karten wurden verteilt
Nachbedingungen	Karte wird umgedreht und ihr Inhalt angezeigt
Standardablauf	1. Spieler wählt eine verdeckte Karte aus 2. System dreht die Karte um und zeigt ihren Inhalt 3. Spieler kann eine zweite Karte umdrehen
Alternative Abläufe	1. Wenn die Karte bereits aufgedeckt ist, passiert nichts 2. Das Spiel kann jederzeit vom Spieler beendet werden

UC4: Paare finden

Aspekt	Beschreibung
Name	Paare finden
Akteur	Spieler
Beschreibung	Der Spieler versucht, zwei passende Karten zu finden
Auslöser	Spieler hat zwei Karten umgedreht
Vorbedingungen	Zwei Karten wurden umgedreht
Nachbedingungen	Bei einem Paar bleiben die Karten aufgedeckt und Punkte werden vergeben, andernfalls werden sie wieder umgedreht

Standardablauf	1. Spieler hat zwei Karten aufgedeckt 2. System prüft, ob die Karten ein Paar bilden 3a. Bei einem Paar: Karten bleiben aufgedeckt, Spieler erhält Punkte 3b. Bei keinem Paar: Karten werden nach kurzer Zeit (2s) wieder umgedreht
Alternative Abläufe	Wenn alle Paare gefunden wurden, wird das Spiel beendet, außerdem in diesem Fass, kann das Spiel auch jederzeit vom Spieler beendet werden

UC5: Punktestand anzeigen

Aspekt	Beschreibung
Name	Punktestand anzeigen
Akteur	System
Beschreibung	Der aktuelle Punktestand des Spielers wird angezeigt
Auslöser	Automatisch während des Spiels
Vorbedingungen	Das Spiel läuft
Nachbedingungen	Punktestand wird aktualisiert
Standardablauf	1. System zeigt den aktuellen Punktestand an 2. Punktestand wird nach jedem gefundenen Paar aktualisiert
Alternative Abläufe	Keine

UC6: Spiel beenden

Aspekt	Beschreibung
Name	Spiel beenden
Akteur	Spieler
Beschreibung	Der Spieler beendet das aktuelle Spiel
Auslöser	Spieler klickt auf Beenden-Button auf Hauptmenü, "zurück zum Menü" während des Spiels oder alle Paare wurden gefunden
Vorbedingungen	Das Spiel läuft
Nachbedingungen	Spiel wird beendet, Ergebnis Bildschirm wird angezeigt

Standardablauf	1. Spiel wird beendet 2. Spieler sieht seinen Endpunktestand 3. Bei einem Highscore kann der Spieler seinen Namen eingeben
Alternative Abläufe	Spieler kann ein neues Spiel starten oder zum Hauptmenü zurückkehren

UC7: Highscores anzeigen

Aspekt	Beschreibung
Name	Highscores anzeigen
Akteur	Spieler
Beschreibung	Der Spieler sieht die Bestenliste
Auslöser	Spieler wählt "Highscores" im Menü oder nach Spielende
Vorbedingungen	Keine
Nachbedingungen	Highscore-Liste wird angezeigt
Standardablauf	1. System zeigt die Bestenlisten für alle Themen und Schwierigkeitsgrade 2. Spieler kann die verschiedenen Listen durchblättern
Alternative Abläufe	Spieler kann zum Hauptmenü zurückkehren

Domain data for the game

Domänenklassen

Klasse	Beschreibung
Spieler	Präsentiert den Benutzer, der das Spiel spielt
Spiel	Der Spielablauf, enthält Spielstand und steuert den Ablauf
Thema	Enthält Informationen zu einem der drei Spielthemen: Mathematik, Englisch und Wörter.
Tutorial	Erklärvideo enthält gesamten Spielablauf und wie das Klick dich schlau gespielt wird

Karte	Eine Spielkarte mit Vorder- und Rückseite,
Paar	Zwei Karten, die zusammen ein gültiges Paar ergeben
Punkttestand	Verwalten der erreichten Punkte pro Spiel.
Ergebnis	Zusammenfassung am Spielende

Beziehungen zwischen den Domänenklassen

- Ein Spieler nimmt an einem Spiel teil.
- Ein Spiel hat ein gewähltes Thema.
- Ein Thema besteht aus mehreren Karten.
- Zwei Karten bilden ein Paar.
- Aktuell gesammelte Paare verwalten den Punkttestand.
- Am Ende eines Spiels wird ein Ergebnis erzeugt.

Context analysis – Project Idea

Context analysis – Vision, Personas

Context analysis – Dictionary of Terms

Begriff	Definition	Typ
Karte	Ein Spielelement, das umgedreht wird, um ein Symbol oder eine Aufgabe zu zeigen.	Spielobjekt
Kartenpaar	Zwei Karten mit dem gleichen Symbol / derselben Aufgabe (für ein Match).	Spielmechanik
Tutorial	Eine Schrittweise Anleitung als Video, das dient dazu, eine bestimmte Fähigkeit oder Funktion zu erklären.	Anleitung
Thema	Kategorie, zu der die Karten gehören (Mathe, Deutsch, Englisch).	Enum (Mathe, Deutsch, Englisch)
Spielrunde	Ein vollständiger Durchlauf des Spiels von Anfang bis Ende.	Prozess
Zug	Aktion eines Spielers: Zwei Karten aufdecken.	Ereignis
Treffer / Match	Zwei Karten mit gleichem Inhalt werden aufgedeckt.	Spielmechanik

Fehlversuch	Zwei aufgedeckte Karten passen nicht zusammen.	Spielmechanik
Punkttestand	Aktueller Stand der richtigen Paare eines Spielers.	Zahl / Score
Timer	Zeitmessung für ein Spiel oder per Zug.	Zeit / Zahl
Mathe-Aufgabe	Inhalt einer Karte im Mathe-Thema (z. B. 3 + 4).	Text / Aufgabe
Deutsch-Wortpaar	Wort und zugehöriges Bild oder Übersetzung (z. B. „Hund“ und Bild).	Text / Bild
Englisch-Vokabelpaar	Englisches Wort und passende Übersetzung oder Bild.	Text / Bild

Context analysis – Quantity Structure

Wichtige Kennzahlen für die Größe unseres Spiels:

1. **Anzahl der Karten:** Für jedes Thema (Mathe, Deutsch, Englisch) müssen wir festlegen, wie viele Karten bzw. Kartenpaare es gibt. Das beeinflusst den Schwierigkeitsgrad und die Spieldauer.
2. **Anzahl der Themen:** Aktuell haben wir 3 Themen, was sich auf die Inhalte, die Gestaltung und die Logik des Spiels auswirkt.
3. **Anzahl der Kartenpaare pro Thema:** Wichtig für den Umfang und die Länge einer Spielrunde.
4. **Anzahl der Spielrunden oder Level:** Wir sollten uns überlegen, ob es bei unserem Spiel unterschiedliche Schwierigkeitsstufen oder Fortschritte gibt.
5. **Audio- und Bildmaterial:** Wie viele Grafiken, Sounds oder Musikstücke müssen erstellt oder integriert werden?
6. **Spielaktionen pro Runde:** Wie viele Interaktionen führt der Spieler aus? Das beeinflusst die Komplexität der Benutzeroberfläche.

Durch die Definition dieser Kennzahlen behalten wir die Kontrolle über den Projektumfang und können sicherstellen, dass das Spiel **in der geplanten Zeit, mit den verfügbaren Ressourcen und in kindgerechter Qualität** umgesetzt wird.

Context analysis – Current State Analysis

Inspiziert vom klassischen Memory, das stark das Kindergedächtnis fördert, entwickeln wir

eine Lernversion für Schulfächer wie Mathe, Deutsch und Englisch. Im Gegensatz zu thematisch oft starren Memorys bieten wir kindgerechte, spielerische Lerninhalte, die durch Updates um neue Stufen oder Fächer erweiterbar werden.

List of Requirements:

Anforderungen sollten systematisch klassifiziert werden. Es handelt sich dabei um:

- FR (Functional requirements)
- NFR (Non-functional requirements)
- Other, non-exclusive categories/labels:
 - T - Requirement for technology
 - L - Legal/contractual requirement
 - UX - Requirements for the user interface
 - A - Requirement for the activities to be carried out
 - C - Requirements for other delivery components
 - Q - Quality requirement

Anforderungen	Must/Can	Kategorien
Kind steuert die Karten	Must	FR, UX, A
Jede Karte hat seine Passende Karte	Must	FR, UX, C
Vokabeln/Mathe muss altersgerecht sein	Must	NFR, Q
Design muss Kinderfreundlich sein	Must	UX, Q
Hintergrundmusik soll spielerisch sein	Can	C, Q
Im Hauptmenü kann Beenden, Start und Tutorial gewählt werden.	Must	FR, UX, A
Im Themenauswahl-Menü können Deutsch, Mathe und Englisch gewählt werden.	Must	FR, UX, A

Priorisierung von nicht-funktionalen Anforderungen

1. Wichtige Qualitätsmerkmale für unser Projekt

Für unser Lernspiel, in dem Erstklässler lesen und rechnen mit Memory lernen, sind drei Sachen besonders wichtig:

- **Einfachheit:** Das Spiel soll so gestaltet sein, dass die Kinder ohne Probleme verstehen, wie es funktioniert.
- **Lernwirkung:** Das Spiel soll den Kindern wirklich helfen, besser lesen und rechnen zu können.
- **Schnelligkeit:** Das Spiel soll flüssig laufen und nicht hängen, damit die Kinder die Lust nicht verlieren.

2. Was das genau bedeutet und wie wir das umsetzen

- **Einfachheit**
Wir gestalten das Spiel bunt und übersichtlich mit großen Bildern und klaren Anweisungen. Die Kinder sollen alles alleine schaffen können, ohne viel Hilfe. Wir testen das, indem wir das Spiel von Kindern ausprobieren lassen und gucken, ob sie es verstehen und bedienen können. Wir haben ein Tutorial erstellt, um den Kindern das Spiel mithilfe eines Videos zu erklären.
- **Lernwirkung**
Unser Memory-Spiel hat kleine Aufgaben zum Lesen und Rechnen, die zum Niveau der Kinder passen. Es gibt leichte und schwerere Aufgaben, damit die Kinder motiviert bleiben. Der Timer und die Versuche sollen die Kinder motivieren, beim nächsten Versuch das Spiel in einer kürzeren Zeit bzw. mit weniger Versuchen zu absolvieren.
Wir messen den Erfolg, indem wir sehen, wie gut die Kinder die Aufgaben lösen und ob sie sich mit der Zeit verbessern.
- **Schnelligkeit**
Das Spiel soll schnell und flüssig laufen und keine langen Ladezeiten haben. Das prüfen wir durch Tests auf verschiedenen Geräten und schauen, ob das Spiel schnell reagiert und stabil bleibt.

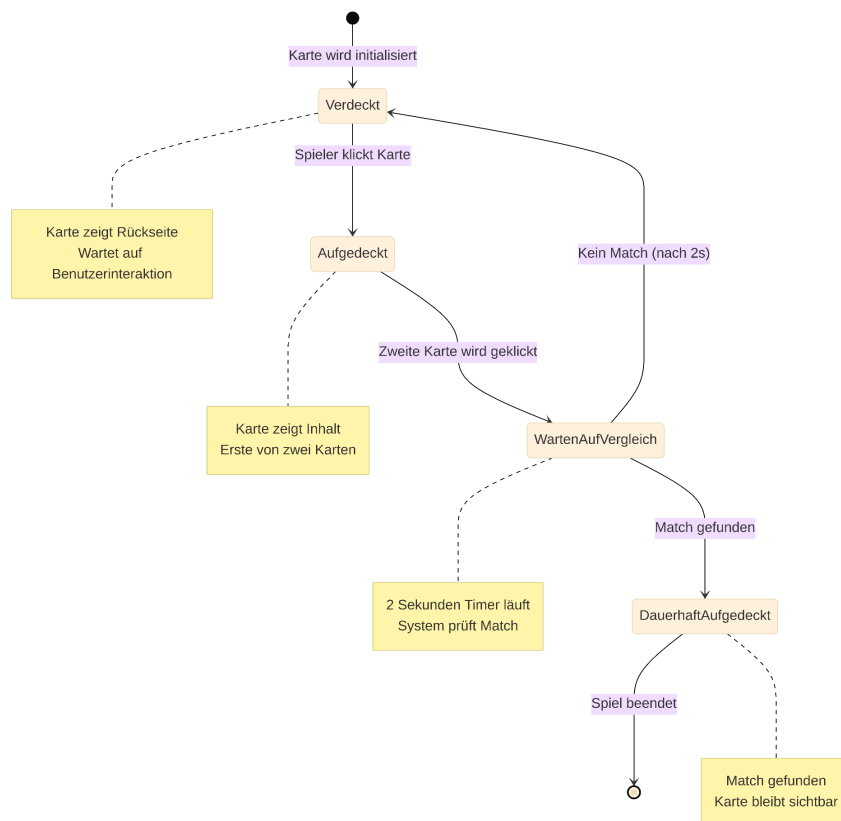
UML Behavior Diagrams

Für unser Memory-Spiel haben wir zwei UML Behavior Diagrams erstellt: ein Zustandsdiagramm für das Kartenverhalten und ein Aktivitätsdiagramm für den Spielablauf.

Zustandsdiagramm - Kartenverhalten

Jede Karte durchläuft vier verschiedene Zustände:

1. **Verdeckt** (Startzustand)
 - Karte zeigt Rückseite
 - Übergang: Mausklick → Aufgedeckt
2. **Aufgedeckt**
 - Karte zeigt Inhalt
 - Übergang: Zweite Karte geklickt → Vergleich
3. **Vergleich**
 - Beide Karten sichtbar für 2 Sekunden
 - Übergang: Match → Gefunden, Kein Match → Verdeckt
4. **Gefunden** (Endzustand)
 - Kartenpaar erfolgreich gefunden
 - Karten bleiben dauerhaft aufgedeckt



Aktivitätsdiagramm - Spielablauf

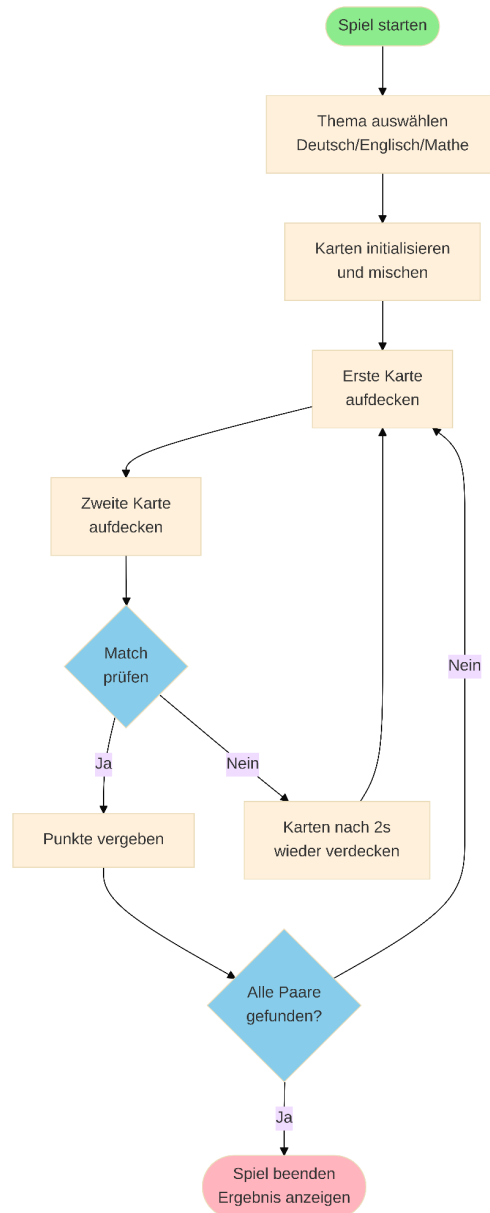
Der Spielablauf besteht aus folgenden Schritten:

1. **Spiel starten**
2. **Thema wählen** (Deutsch, Englisch oder Mathe)
3. **Karten mischen und verteilen**

4. Spielschleife:

- Erste Karte aufdecken
- Zweite Karte aufdecken
- Match prüfen
- Falls Match: Punkte vergeben
- Falls kein Match: Karten wieder verdecken

5. Spiel beenden (alle Paare gefunden)



Begründung:

Zustandsdiagramm: Geeignet für Objekte mit klaren Zuständen und Übergängen

Aktivitätsdiagramm: Zeigt Workflow und Entscheidungspunkte im Spielablauf

Die Diagramme helfen bei der Implementierung in Godot und sorgen für eine strukturierte Code-Entwicklung.

02 Architectural documentation

Description of the system architecture

Functional and non-functional requirements (reference to requirements documentation)

Funktionale Anforderungen:

- Das Spiel zeigt ein Raster mit verdeckten Karten.
- Ein Spieler darf zwei Karten pro Runde umdrehen.
- Bei Übereinstimmung bleiben Karten aufgedeckt.
- Anzeige von Punkten und Anzahl der Züge.
- Bei allen Paaren aufgedeckt: Siegescreen.

Nicht-funktionale Anforderungen:

- **Kindgerechte Bedienung:** Große Buttons, einfache Sprache.
- **Barrierefreiheit:** Kontraste, einfache Schriftarten.
- **Performance:** Flüssiger Ablauf auch auf schwachen Geräten.
- **Skalierbarkeit:** Einfaches Hinzufügen neuer Levels oder Designs.
- **Fehlertoleranz:** Keine Abstürze bei schnellen Klicks.

Prioritization of non-functional requirements

Anforderung	Priorität	Begründung
Benutzerfreundlichkeit	Hoch	Zielgruppe sind Kinder – intuitive Steuerung nötig
Performance	Hoch	Muss auch auf älteren Tablets laufen
Barrierefreiheit	Mittel	Unterstützung für Sehschwächen wichtig, aber nicht kritisch
Erweiterbarkeit	Gering	Wünschenswert, aber nicht primär

Architectural principles

Kapselung: Jede Szene kapselt ihre eigene Logik und Darstellung.

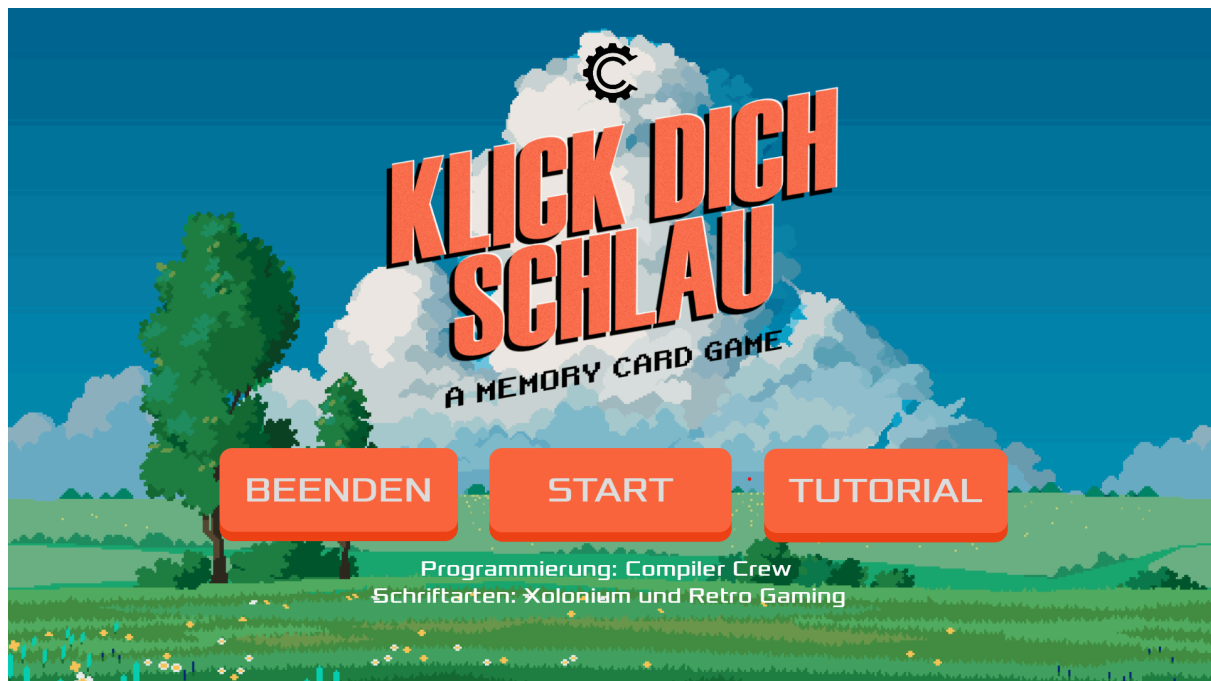
Lose Kopplung: Kommunikation über Signale anstelle direkter Abhängigkeiten.

Single Responsibility Principle (SRP): Jeder Node hat eine klar definierte Aufgabe.

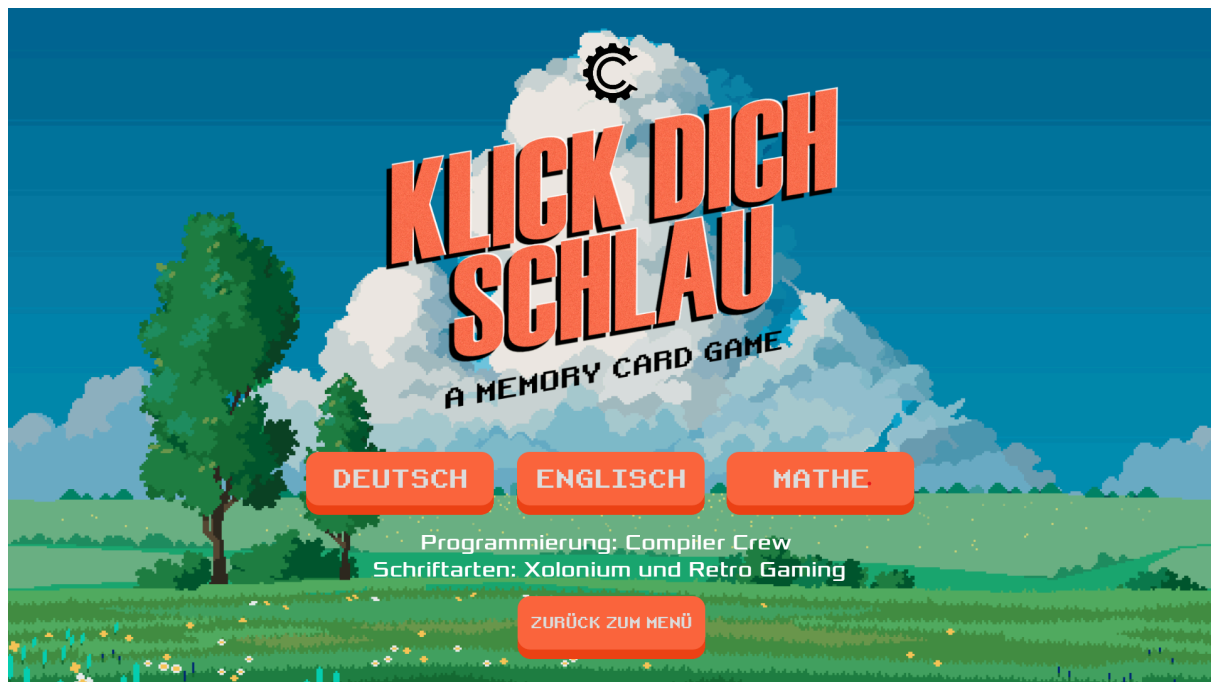
Modularität: Wiederverwendbare Komponenten (z. B. Karten-Logik unabhängig vom Layout).

Kindgerechtes UI-Design: Farbwahl, einfache Animationen, Touch-Freundlichkeit.

Interfaces



Hauptmenü



Themenauswahl-Menü

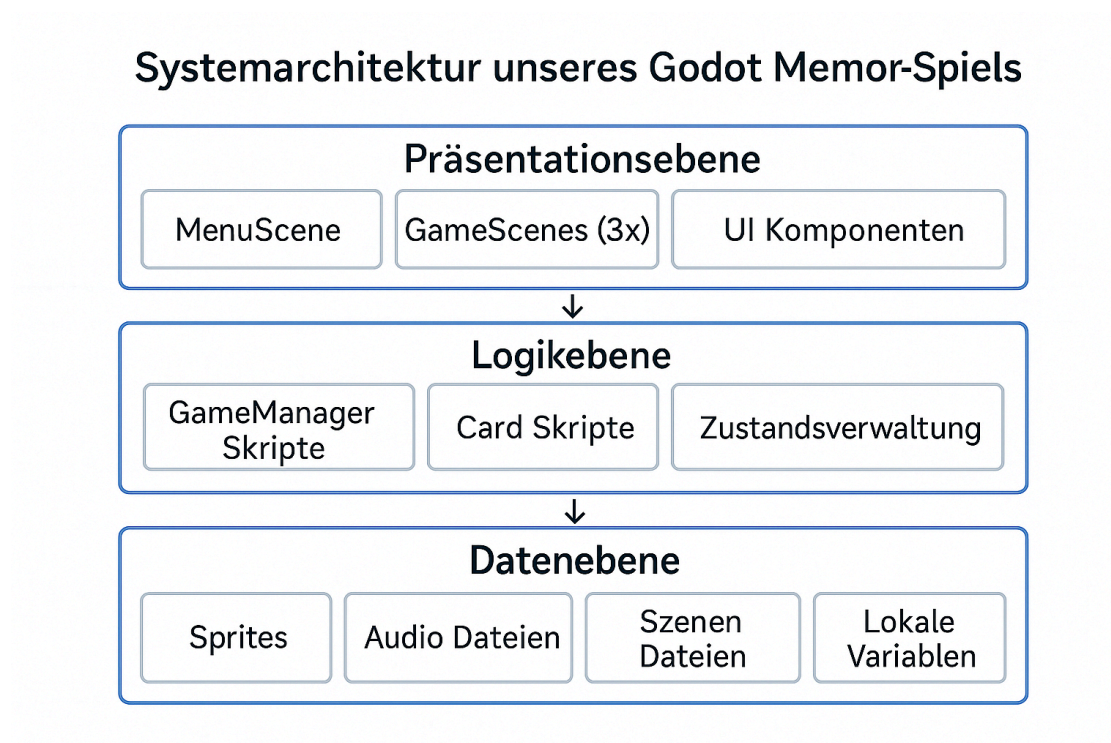


Spielbildschirm (Thema: Mathe)

Big picture of the system architecture

Unser "Klick dich schlau" Memory-Spiel folgt einer klassischen 3-Schichten-Architektur.

Architektur-Übersicht:



Unser System ist in drei klar getrennte Ebenen organisiert:

1. Präsentationsebene (UI Layer)

- MenuScene: Hauptmenü und Menü der Themenauswahl
- GameScenes (3x): Separate Spielszenen für Deutsch, Englisch und Mathe
- UI Components: Timer, Punkteanzeige, Kartenbezeichnungen
- Audio Player: MatchSoundPlayer für Erfolgs-Sounds

2. Logikebene (Game Logic Layer)

- GameManager Scripts: Drei separate Skripte für jedes Thema
- Card Scripts: Kartenverhalten und Klick-Erkennung
- State Management: Spielzustand, Zeitverlauf und Paarprüfung
- Navigation: Szenen-Wechsel zwischen Menü und Spielmodi

3. Datenebene (Asset Layer)

- Sprites: Themenspezifische Ordner mit Kartenbildern
- Audio Files: 3 Dateien (2x MP3 Musik, 1x WAV Match-Sound)
- Scene Files: 6 .tscn-Dateien für UI-Layouts
- Local Variables: Spielstand und Kartenzustände

Architektur-Prinzipien

- Separation of Concerns: Jede Schicht hat klar definierte Verantwortlichkeiten
- Modularität: Jedes Thema ist eigenständig implementiert
- Einfachheit: Keine komplexen Design Patterns
- Godot-Konformität: Nutzt szenen-basierte Organisation der Engine

Der Datenfluss ist linear und nachvollziehbar:

1. Benutzer-Input (Mausklick) → Präsentationsebene
2. Präsentationsebene → Logikebene (GameManager)
3. Logikebene → Datenebene (Zugriff auf Assets)
4. Datenebene → Logikebene → Präsentationsebene (UI-Update)

Diese Architektur konzentriert sich auf Verständlichkeit und Wartbarkeit - ideal für ein studentisches Lernspiel-Projekt.

System design

Unser Memory-Spiel folgt einer einfachen, szenen-basierten Architektur, die typisch für Godot-Projekte ist.

Grundlegende Architektur

1. Szenen-basierte Organisation
 - MenuScene: Hauptmenü mit Themenauswahl
 - 3 GameManager-Szenen: Separate Spiellogik für jedes Thema (Deutsch, Englisch, Mathe)
 - Card-Komponenten: Wiederverwendbare Kartenelemente für jedes Thema
2. Themenspezifische Struktur

- Drei separate GameManager-Skripte mit eigener Spiellogik
 - Themenspezifische Card-Skripte (Card.gd, card_english.gd, card_math.gd)
 - Getrennte Asset-Ordner für jedes Thema
3. Einfache Datenorganisation
 - Lokale Variablen in GameManager für Spielzustand
 - Preloading von Texturen für bessere Performance
 - Direkte Kommunikation zwischen Karten und GameManager

Designprinzipien

1. Einfachheit
 - Klare Trennung der drei Themen
 - Minimale Abhängigkeiten zwischen Komponenten
 - Verständliche Code-Struktur ohne komplexe Patterns
2. Wiederverwendbarkeit
 - Ähnliche Struktur für alle drei GameManager
 - Identische Card-Funktionalitäten mit themenspezifischen Sprites
 - Konsistente Asset-Organisation
3. Erweiterbarkeit
 - Neue Themen können durch Kopieren und Anpassen hinzugefügt werden
 - Modulare Struktur ermöglicht einfache Änderungen
 - Klare Trennung von Spiellogik und Inhalten

Diese Architektur ist bewusst einfach gehalten und konzentriert sich auf die Kernfunktionalität des Memory-Spiels, ohne unnötige Komplexität.

System decomposition

Das System besteht aus überschaubaren, klar abgegrenzten Komponenten, die der Godot-Projektstruktur folgen.

Hauptkomponenten:

Navigation und Menü

- MenuScene.gd: Hauptmenü mit Start-Button und Themenauswahl
- themes.tscn: Themenauswahl-Szene mit drei Buttons
- Button-Callbacks für Szenenwechsel zwischen Menü und Spielmodi

Spiellogik-Module (Zentrale Steuerung)

- GameManager_Deutsch.gd: Verwaltet deutsche Wörter-Paare (12 Paare)
- GameManager_English.gd: Verwaltet englische Begriffe mit identischer Struktur
- GameManager_Math.gd: Verwaltet mathematische Aufgaben-Paare
- Jeder GameManager: Speichert 24 Bildvariablen, Paar-Namen, Timer und Versuche-Zähler
- Zentrale Spiellogik: Paarprüfung, Spielstand-Verwaltung, Statistik-Tracking

Karten-System (Interaktions-Ebene)

- Card.gd: Kartenverhalten für deutsche Karten (Area2D Input-Events)
- card_english.gd: Identisches Kartenverhalten für englische Karten
- card_math.gd: Identisches Kartenverhalten für Mathe-Karten
- Alle Card-Skripte: Klick-Erkennung, Texturwechsel, Kommunikation mit GameManager
- Beziehung: Cards empfangen Klicks → senden Info an GameManager → GameManager entscheidet

UI-Komponenten

- Timer-System: Spielzeit-Messung direkt in GameManager-Variablen
- Versuche-Zähler: Statistik-Tracking in GameManager

Asset-Organisation

- sprites/: Themenspezifische Ordner (german/, english/, math/) mit je 12 Bildpaaren
- Sound/: 3 Audio-Dateien (2x MP3 Hintergrundmusik, 1x WAV Match-Sound)
- scenes/: 6 Szenen-Dateien (.tscn) für verschiedene Spielbereiche

Audio-System

- MatchSoundPlayer: Erfolgs-Feedback bei gefundenen Paaren
- Einfache Sound-Wiedergabe ohne komplexe Audio-Manager-Struktur

Komponentenbeziehungen

- GameManager steuert Card-Instanzen direkt
- Cards kommunizieren über on_click() zurück an GameManager
- Assets werden beim _ready() geladen
- UI-Elemente werden direkt über Variablen aktualisiert

Die Struktur ist linear und gut nachvollziehbar - genau richtig für ein Memory-Spiel dieser Größe.

Design alternatives and decisions

Bei der Entwicklung haben wir pragmatische Entscheidungen getroffen, die für ein studentisches Memory-Spiel angemessen sind.

Wichtige Designentscheidungen

1. Engine-Wahl: Godot

- Begründung: Kostenlos, gut für 2D-Spiele, einfach zu erlernen
- Alternative: Unity (zu komplex und kostenpflichtig für unser Projekt)
- Vorteil: Integrierte Szenen-Verwaltung und einfache Asset-Pipeline
- (Voraussetzung für die Prüfung)

2. Programmiersprache: GDScript

- Begründung: Native Godot-Sprache, Python-ähnliche Syntax
- Alternative: C# (unnötig komplex für Memory-Spiel)
- Vorteil: Einfache Integration mit Godot-Features, schnell zu erlernen

3. Themen-Organisation: Separate GameManager

- Begründung: Einfache Trennung, jeder kann an eigenem Thema arbeiten
- Alternative: Ein GameManager mit Themen-Parameter (komplexer zu programmieren)
- Vorteil: Klare Verantwortlichkeiten, einfache Parallelarbeit im Team

4. Kartenlogik: Direkte Kommunikation

- Begründung: Einfach zu verstehen und zu debuggen
- Alternative: Event-System mit Signals (Overkill für Memory-Spiel)
- Vorteil: Minimaler Code-Overhead, direkte Kontrolle

5. Asset-Struktur: Ordner-basiert

- Begründung: Übersichtliche Organisation nach Themen
- Alternative: Zentrale Asset-Verwaltung (unnötig komplex)
- Vorteil: Einfaches Hinzufügen neuer Inhalte, klare Struktur

6. Audio-Implementation: Einfache Player

- Begründung: Ausreichend für unsere Anforderungen
- Alternative: Audio-Manager-System (überdimensioniert)
- Vorteil: Direkte Kontrolle, wenig Code, funktioniert zuverlässig

Bewertung der Entscheidungen Alle Entscheidungen zielen auf Einfachheit und Verständlichkeit ab. Für ein Lernspiel mit 3 Themen und 12 Kartenpaaren sind komplexere Architekturen nicht nötig.

Die gewählten Lösungen ermöglichen:

- Schnelle Entwicklung im Team
- Einfache Wartung und Debugging
- Verständlichen Code für alle Teammitglieder
- Erfolgreiche Umsetzung in der verfügbaren Zeit

Cross-cutting concerns, non-functional requirements (NFA or NFR)

Folgende querschnittliche Belange und nicht-funktionale Anforderungen haben wir berücksichtigt:

Performance

- Anforderung: Flüssige Darstellung (60 FPS)
- Umsetzung: Optimierte Texturen, einfache Preloading-Struktur
- Messung: FPS-Counter im Debug-Modus

Usability

- Anforderung: Intuitive Bedienung für Kinder ab 6 Jahren
- Umsetzung: Große Buttons, klare Symbole, einfache Navigation
- Messung: Usability-Tests mit Zielgruppe

Accessibility

- Anforderung: Barrierefreie Nutzung
- Umsetzung: Hohe Kontraste, Schriftgrößen anpassbar
- Messung: Kontrast-Checker, Feedback von Nutzern

Portability

- Anforderung: Läuft auf Windows, macOS, Linux
- Umsetzung: Godot Engine (plattformübergreifend)
- Messung: Tests auf allen Zielplattformen

Maintainability

- Anforderung: Code soll erweiterbar und wartbar sein
- Umsetzung: Modulare Architektur, Kommentierung, Coding Standards
- Messung: Code-Reviews, Dokumentationsgrad

Security

- Anforderung: Keine sensiblen Daten, kindersicher
- Umsetzung: Lokale Datenspeicherung, keine Netzwerkverbindungen
- Messung: Security-Audit, Datenschutz-Prüfung

Reliability

- Anforderung: Stabile Ausführung ohne Abstürze
- Umsetzung: Exception Handling, Input Validation
- Messung: Stress-Tests, Error-Logging

Scalability

- Anforderung: Einfache Erweiterung um neue Themen
- Umsetzung: Modulare Themen-Struktur mit separaten GameManagern
- Messung: Zeit für Hinzufügung neuer Inhalte

Diese NFRs werden kontinuierlich überwacht und bei Bedarf angepasst.

Human-machine interface

Klick dich schlau, nutzt einfache, aber effektive Eingabe- und Ausgabemethoden.

Eingabemethoden

1. Maus-Interaktion
 - Linksklick auf Karten zum Aufdecken
 - Klicken auf Buttons für Navigation

- Point-and-Click Bedienung im gesamten Spiel

Ausgabemethoden

1. Visuelle Ausgabe
 - Kartenanimationen beim Aufdecken (Texturwechsel)
 - UI-Elemente: Punkteanzeige, Timer, Kartenbezeichnungen
 - Themenspezifische Grafiken (Deutsch, Englisch, Mathe)
 - Farbliche Unterscheidung der Themen
2. Audioausgabe
 - Soundeffekt bei erfolgreichen Matches (MatchSoundPlayer)
 - Hintergrundmusik (fairy-tale-loop, game-music-loop)
 - Erfolgs-Sound bei Paarfindung

Interaktionsparadigmen

- Point-and-Click: Ausschließliche Maussteuerung
- Direct Manipulation: Direkte Kartenauswahl durch Klick
- Feedback-orientiert: Sofortiges visuelles Feedback beim Kartenaufdecken
- Sequenzielle Interaktion: Erst Karte 1, dann Karte 2, dann automatische Auswertung

Die Schnittstelle ist bewusst einfach gehalten und konzentriert sich auf die Kernfunktionalität des Memory-Spiels.

Requirements for the Human-machine interface

Folgende Anforderungen haben wir für die Benutzeroberfläche umgesetzt:

Usability-Anforderungen

1. Einfachheit
 - Nur eine Eingabemethode: Mausklick
 - Selbsterklärende Karteninteraktion
 - Klare Themenauswahl über Buttons
2. Reaktionszeit
 - Sofortige Kartenaufdeckung bei Klick
 - Automatische Paarprüfung nach 2. Karte
 - Flüssige Texturwechsel
3. Konsistenz
 - Einheitliche Klick-Interaktion in allen Spielmodi
 - Gleiche Button-Bedienung über alle Szenen
 - Konsistente Kartengrößen und -positionen

Funktionale Anforderungen

1. Karteninteraktion
 - Karten können einzeln angeklickt werden
 - Aufgedeckte Karten zeigen Inhalt (Bild/Text)
 - Automatische Rücksetzung bei Nicht-Match

2. Audio-Feedback
 - Sound bei erfolgreichen Matches
 - Hintergrundmusik während des Spiels
 - Audio-Dateien: MP3 und WAV Format
3. Themenunterstützung
 - Drei verschiedene Spielmodi (Deutsch, Englisch, Mathe)
 - Themenspezifische Inhalte und Grafiken
 - Separate GameManager für jeden Modus

Performance-Anforderungen

- Godot Engine Performance
- Unterstützung für Desktop-Plattformen
- Lokale Asset-Verwaltung ohne Netzwerk

Design principles and style guide

Unsere Gestaltung folgt dem Prinzip der Einfachheit und Klarheit.

Designprinzipien

1. Einfachheit
 - Reduzierte Benutzeroberfläche
 - Fokus auf Kartenspiel-Mechanik
 - Keine überflüssigen UI-Elemente
2. Klarheit
 - Deutlich erkennbare Karten
 - Klare Themenunterscheidung
 - Einfache Navigation zwischen Szenen
3. Konsistenz
 - Einheitliche Kartengrößen
 - Gleiche Interaktionsmuster
 - Konsistente Button-Platzierung

Themenspezifische Farbgebung:

- Deutsch: Traditionelle deutsche Farben
- Englisch: Eigene Farbpalette
- Mathe: Mathematik-orientierte Gestaltung

Layout-Prinzipien

1. Kartenraster
 - Gleichmäßige Verteilung der Karten
 - Ausreichend Abstand zwischen Karten
 - Zentrierte Anordnung
2. UI-Platzierung
 - Timer und Punktestand unten
 - Kartenbezeichnungen unten
 - Navigation-Buttons an gewohnten Positionen

Asset-Verwaltung

- Sprites-Ordner mit themenspezifischen Unterordnern
- Sound-Ordner mit MP3/WAV-Dateien
- Szenen-basierte Organisation

Interaction modeling

Die Interaktionsabläufe sind bewusst einfach und linear gestaltet.

Hauptinteraktions-Workflow

1. Spielstart
2. Kartenspiel-Workflow
3. Paarprüfung

State-Management

1. Kartenzustände
 - Verdeckt (card_back Textur)
 - Aufgedeckt (card_face Textur)
 - Gefunden (number_of_matches)
2. Spielzustände
 - Warten auf erste Karte
 - Warten auf zweite Karte
 - Paarprüfung läuft
 - Spiel beendet (alle Paare gefunden)
3. Audio-Zustände
 - Hintergrundmusik läuft
 - Match-Sound wird abgespielt
 - Stille zwischen Aktionen

Feedback-Mechanismen

1. Visuelles Feedback
 - Sofortige Texturänderung bei Kartenklick
 - Kartenbezeichnungen werden aktualisiert
 - Timer läuft sichtbar mit
2. Audio-Feedback
 - MatchSoundPlayer spielt bei Erfolg
 - Hintergrundmusik für Atmosphäre
 - Keine Fehler-Sounds (bewusst positiv)

Error-Handling

- Karten können nicht doppelt geklickt werden (click_enabled Flag)
- Automatische Rücksetzung nach Nicht-Match
- Robuste Szenen-Navigation

Die Interaktion ist darauf ausgelegt, dass Spieler intuitiv verstehen, was zu tun ist, ohne komplexe Erklärungen zu benötigen.

03 Test documentation

Test specification

Testobjekte: Spielerfiguren, Karten, Sound, Funktion der Karten (aufdecken, zu decken, effekt wenn gleiches paar/ ungleiches paar aufgedeckt wird)

Testziel: Reibungsloser verlauf, Kindergerecht

Testmethoden: WELCHE NEHMEN WIR

Testumgebung: Software

Test protocol

Datum und Uhrzeit der Testdurchführung

Wer hat den Test durchgeführt:

Was wird getestet:

Testergebnis:

Abweichungen von der Erwartung:

vt Screenshots?

04 Acceptance documentation

SUT - System Under Test

BZA - Provision for acceptance

Das ist die **formale Vorbereitung und Übergabe** des Systems an den Kunden oder die Abnahmeinstanz. Du erklärst damit:

„Das System ist bereit zur Abnahme – bitte testen Sie es anhand der vereinbarten Kriterien.“

Typische Inhalte:

- Installationspakete, Zugangsdaten
- Anleitung zur Testdurchführung
- Liste der Abnahmekriterien (z. B. Anforderungsdokumente, vereinbarte Use Cases)

Submission of acceptance report incl. agreed use cases

05 User documentation

Project Approach:

Vorgehensmodell:

Phasen: Beispiel: Anforderungsanalyse → Entwurf → Implementierung → Test → Abnahme

Werkzeuge: Google Docs, WhatsApp Abstimmungen für Aufgabenverteilung

Teamstruktur und Rollen:

Klare Aufgabenverteilung: Ibrahim erstellt vor jedem Sprint eine Übersicht, wer was übernimmt.

Bessere Kommunikation: Sascha sorgt dafür, dass wir 1x pro Woche ein kurzes Team-Meeting machen.

Zwischenziele setzen: Iman teilt große Aufgaben in kleine Schritte ein und dokumentiert sie.

Technische Dokumente ergänzen: Ian und Luca schreiben nach jedem Sprint eine kurze Zusammenfassung, was programmiert wurde.

Kommunikation & Meetings:

Lessons Learned

gemeinsam als Team durchgehen

06 Technische Dokumentation

Technische Doku:

1 Sprint: 24.04 - 09.05:

- Ersten Projektaufgaben erledigt (Kanban Board)
- Spielideen gesammelt

2 Sprint: 09.05 - 23.05

- Wir haben uns für eine Spielidee entschieden und ein grobes Brainstorming gemacht was man so implementieren kann
- Prototyp, Sales Pitch, Visionboard

3 Sprint: 23.05 - 06.06

- Angefangen mit der Entwicklung in GoDot
- Deutsch Englisch und Mathe erstellt

4 Sprint: 06.06 - 20.06

- Sound implementiert
- Feinschliff der Karten etc.)
- Timer und Versuche hinzugefügt
- Tutorial erstellt

5 Sprint: 20.06 - 04.07

- Dokumentation gemeinsam verbessert
- Testing